CHATBOT
Rata Mania - CyberRat

Jhon Farfan

Diana Pacheco

Documentación

En este documento se describirán los puntos claves para el despliegue y administración del chatbot, desde sus dependencias hasta la forma de interactuar directamente con el código y la base de datos en sqlite.

Despliegue

Para encender la App de forma correcta hay que tener docker funcionando sin problemas, ya sea en una distribución linux, windows o mac. Esto con el fin de que este proyecto corra de igual forma dentro de cualquier sistema operativo.

Al leer el archivo Dockerfile, verá que hay dos servicios principales: **api** y **gradio**. Es importante que lo tenga en cuenta, ya que ambos son desplegados por separado, pero gracias a la familia de comandos de “docker compose” con una sola instrucción todos los servicios serán levantados.

```
cd docker
echo "GROQ_API_KEY=<API_KEY>" > .env
echo "GROQ_MODEL=openai/gpt-oss-120b" >> .env # otro modelo si lo prefiere
docker compose up --build
```

Hasta acá lo que ha hecho es descargar y construir toda la aplicación de forma segura haga esto solo como el primer paso de iniciación, después no será necesario ejecutar exactamente estas mismas líneas si ya lo hizo una vez.

EN OTRA TERMINAL - Asegúrese de estar dentro del directorio docker

```
docker compose exec -it <nombre del servicio> bash
```

Este será el comando para entrar en el contenedor de alguno de los dos servicios.

Teniendo en cuenta el servicio api y el gradio, estas serán las formas de entrar

```
docker compose exec -it api bash # servicio de Fast API
```

```
docker compose exec -it gradio bash # servicio gradio web
```

(Si ya desplegó todo también puede entrar en su navegador a http://localhost:7860 Para entrar al chat de gradio)

Administración

Una vez haya ejecutado los últimos docker compose exec para administrar el chat, ver la base de datos u otra razón, hay que tener en cuenta que

1. Los contenedores están basados en `FROM` python:3.12-slim que es una versión ligera de Debian.
2. Al ser slim, significa que es un contenedor con solo lo necesario, así que todo lo que este descargado alli saldra de parte del archivo requirements.txt
3. Una vez ejecute alguno de los dos comandos entrara a una shell

Dejando eso claro, administrar desde la consola necesitará al menos de **nano**, **vim**, **sqlite3....** y cualquier otra utilidad que necesite, para eso primero habrá que actualizar el gestor de paquetes:

```
root@c8da51cb1394:/app# apt-get update
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4
kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9678 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [720
```

```
B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64
Packages [270 kB]
Fetched 10.0 MB in 12s (846 kB/s)
Reading package lists... Done
```

Y ya a partir de aca podra usar comandos normales de linux, como el apt-get install <nombre del paquete> y demás.

```
root@c8da51cb1394:/app# ls
README.md       consultaa.py   gradio_e.py probar_demo.py ratas.db scripts
RataMania.json     docker           main.py     prompt_sistema.txt
requirements.txt   test_consulta.py
__pycache__ docs   negocio.json  puente_flowise.js    reservas.json
```

Por ejemplo, acá un simple "ls" muestra todos los archivos que componen el proyecto.

Apagar Servicios

Para ya cerrar todo podrá ejecutar Ctrl + c en la consola que ejecutó el primer docker compose, pero hay otras formas de hacerlo:

(dentro del mismo directorio docker)

```
docker compose down
```

Al ejecutar esto estará eliminando y deteniendo los contenedores, la base de datos y el código estarán intactos así hayan tenido cambios, lo que sí cambiará serán las configuraciones y dependencias que haya descargado manualmente.

```
docker compose stop
```

Esto lo que hará será detener los contenedores, pero a diferencia del down, este mantendrá los volúmenes y contenedores intactos.

Encender servicios de nuevo

Aca, simplemente los podrá volver a encender dependiendo de cómo los haya apagado, de nuevo, desde el directorio **docker**.

```
docker compose up
```

Este solo si ejecuto “docker compose down” y ya no necesitará el parámetro “--build” de nuevo (a menos que haga cambios significativos en el código)

```
docker compose start
```

Este solo si ejecuto “docker compose stop”, solamente restablece los servicios, y usted podrá entrar de nuevo a la shell con “docker compose exec -it…” y todo lo configurado allí dentro se mantendrá.